



중간 발표

프로젝트: MoonShot

파트3 5팀

팀원: 김지수, 나영준, 박건용, 이광수, 이상휘

▼ 프로젝트 개요

목적/ 한 줄 소개

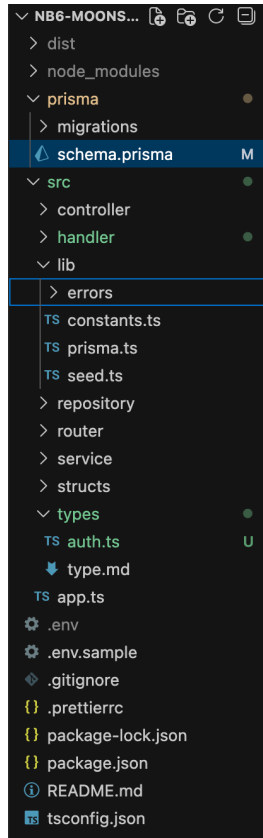
- 제목: Moonshot
- 소개: 프로젝트 일정 관리 서비스

핵심 기능

- 유저, 프로젝트, 할 일, 하위 할 일, 댓글, 멤버 등의 CRUD 작업
- 인증 및 구글 소셜 로그인

현재 진행 단계

- 프로젝트 설계 완료



- 구현 단계 진행 중

▼ 개발 환경 및 협업 규칙

기술 스택

분류	사용 예정 도구
Backend	Node.js (Express), TypeScript
Database	postgresql
API 문서화	Swagger
협업 도구	Discord, GitHub, Notion
일정 관리	GitHub Issues + Notion

협업 규칙

항목	내용
네이밍 컨벤션	camelCase (변수, 함수), PascalCase (클래스), kebab-case (파일)
커밋 컨벤션	[생성] userController 함수, [수정] auth 기능 / [큰 틀] 함수이름(기능)
브랜치 전략	main , feature/기능명 , bugfix/이슈번호
PR 규칙	2명 이상 Approve 시 Merge

▼ 스키마 설계 개요

주요 테이블

User 유저				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
이메일	email	String		Unique, Required
이름	name	String		Required
비밀번호	password	String?		Nullable (OAuth 사용하면 null)
프로필 이미지	profileImage	String?		Nullable
인증 제공자	provider			Nullable, Unique
OAuth 제공자 ID (ex 구글숫자키)	providerId			
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
작성한 댓글	comments	Comment[]		
Project/프로젝트				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
프로젝트 이름	name	String		Required
설명	description	String?		Nullable
생성자 ID (User id)	ownerId			FK → User.id
API 응답시 계산 가능함	memberCount	Number		
API 응답시 계산 가능함	todoCount	Number		
API 응답시 계산 가능함	inProgressCount	Number		
API 응답시 계산 가능함	doneCount	Number		
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
ProjectMember / 프로젝트 멤버				
name	type		제약조건	
고유값	id	Int		PK, Auto Increment
프로젝트 ID	projectId	Int		FK → Project.id
사용자 ID	userId	Int		FK → User.id
생성시간	createdAt	DateTime		
Invitation/초대				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK
프로젝트 ID	projectId	Int		FK → Project.id
초대받은 이메일	invitedEmail	String		Required
초대상태	status	Enum		pending/accepted/rejected 중 1
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
Attachment / 첨부파일				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
파일 URL	url	String		Required
할일 ID	taskId	Int		FK → Task.id
Cascade 삭제 규칙				
삭제 대상	함께 삭제해야하는 데이터			
User 삭제	ProjectMember, Comment			
Project 삭제	ProjectMember, Task, Invitation			
Task 삭제	SubTask, Comment, TaskTag, Attachment			
Tag 삭제	TaskTag (만일만 삭제)			
트랜잭션 체크				
프로젝트 생성	Project 생성 + ProjectMember에 생성자(owner) 자동등록			
초대 수락	Invitation 상태 업데이트 + ProjectMember 추가			
프로젝트 삭제	관련 데이터 삭제 + 알림			

Task/할 일				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
제목	title	String		Required
프로젝트 ID	projectId	Int		FK → Project.id
시작년	startYear	Int		Required
시작월	startMonth	Int		Required
시작일	startDay	Int		Required
종년	endYear	Int		Required
종월	endMonth	Int		Required
종일	endDay	Int		Required
진행상태	status	Enum	enum ["todo", "in_progress", "done"]	
담당자	assigneeId	Int?		FK → User.id, Nullable
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
SubTask/하위 할 일				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
제목	title	String		Required
상위 할 일	taskId	Int		FK → Task.id
상태	status	Enum	enum ["todo", "in_progress", "done"]	
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
Comments/댓글				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
내용	content	String		Required
할일 ID	taskId	Int		FK → Task.id
작성자 ID	authorId	Int		FK → User.id
생성시간	createdAt	DateTime	@default(now())	
수정시간	updatedAt	DateTime	@updatedAt	
Tag / 태그				
name	type		제약조건	
고유값	id	Int	@id @default(autoincrement())	PK, Auto Increment
태그 이름	name	String		Unique, Required
TaskTag / 할일-태그 연결				
name	type		제약조건	
할일 ID	taskId	Int	@id @default(autoincrement())	PK, FK → Task.id
태그 ID	tagId	Int		PK, FK → Tag.id

하나의 할 일에 여러 태그, 하나의 태그에 여러 할 일 (다태그를 위한 중간 테이블)

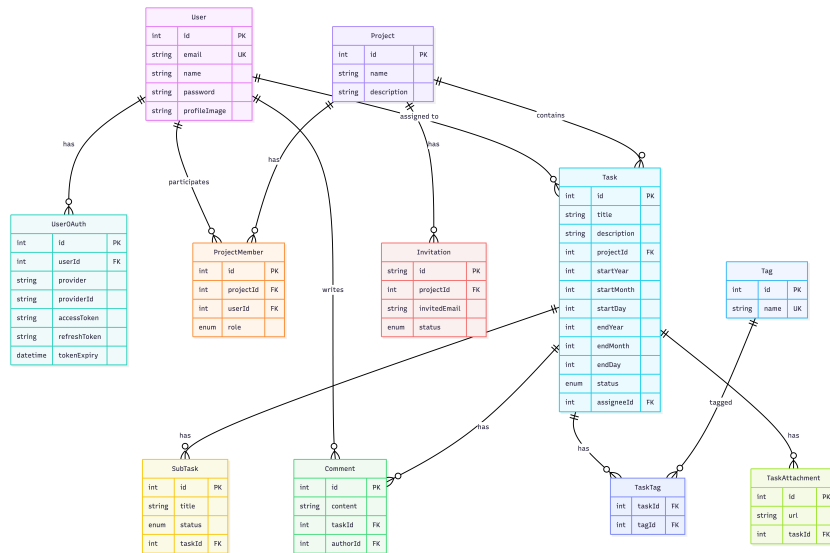
- 구글 스프레드시트를 이용해 최초 설계

설계 시 고민 포인트

- N:M 관계 해결책
 - 태그의 경우 할 일 스키마의 필드로 넣을 수도 있고 태그 스키마를 만들어 따로 관리할 수도 있음

▼ ERD

ERD 전체 구조



• 스키마 변동에 따라 변동 예정

핵심 엔티티 관계

- 1:N 관계
 - 유저와 댓글, 유저와 할 일, 프로젝트와 할 일, 할 일과 첨부파일 등
- N:M 관계
 - 유저와 프로젝트 ⇒ 중간 테이블 ProjectMember 생성
 - 할 일과 태그 ⇒ 중간 테이블 TaskTag 생성

관계 설정 이유

- 1:N 관계
 - 하나의 유저는 여러개의 댓글을 작성할 수 있으며 하나의 댓글은 반드시 하나의 유저를 가짐
 - 데이터 소유 관계를 명확히 하기 위함
- N:M 관계
 - 하나의 유저가 여러 프로젝트에 참여할 수 있으며 여러 유저가 하나의 프로젝트에 참여 가능함
 - N:M 관계를 직접 표현할 수 없어 중간 테이블을 만들어 관리
- 엔티티 간 책임을 명확히 하고, 협업 구조 확장과 데이터 무결성을 동시에 고려한 관계 설계

▼ R&R

- 공통 스키마 설계 → 역할 분담
- 유저를 제외한 api 분배 ⇒ 마지막에 유저를 통해 모든 과정 검증
- 이후 유저 및 추가 구현 사항 등 진행상황에 따라 추가될 예정

팀원	주요 담당
김지수	댓글
나영준	프로젝트 설계 / 인증 / 파일 업로드
박건용	프로젝트 / 멤버
이광수	하위 할 일
이상휘	할 일

▼ 멘토님 피드백

- 진행 중
 - token expiry는 꼭 필요한지 → 필요하다면 어떤 토큰의 만료일자인지
 - userOauth 에서 토큰은 서비스 내의 토큰인지 구글의 토큰인지
 - → 구글 토큰이라면 기존 이메일/패스워드 로그인 시 토큰은 어떻게 저장되는지 → **userToken 테이블을 별도로 만들어야 하는 것은 아닐까**
- 완료
 - 초대 스키마에서 초대받은 이메일인 이유 → **명세서 따라서 진행해 이메일로 설정**
 - 기존 이메일 패스워드로 가입한 유저가 구글로 로그인 시 동일한 유저로 연결해주는 것인지 다른 유저로 인식하는지. → **나중에 시간이 남는다면 발전**
 - 첨부파일 이름 → **url의 뒷부분으로 가능하긴 함**
 - TaskAttachment로 바꾸는게 더 정확한 이름이지 않을까 → **TaskAttachment로 설정**
 - user에서 name → **api 명세서에서 name으로 사용중이므로 유지**
 - 할 일에 내용 넣을 필드 → **명세서에는 없으나 디자인 시안에 내용을 넣는 공간이 있음 description 생성**
 - 태그 스키마를 굳이 해야하나..?(주로 태그를 통해 검색을 할 때 다대다로 많이 나눔) ⇒ **초급 프로젝트에도 비슷한 상황이있었음 → 이번엔 다대다로 해봄 → 하다가 애매하다 싶으면 바꿀 예정**
 - startDate, endDate로 바꾸면 되지 않을까 → **api 명세서에 따르기로**

▼ 현재 프로젝트 진행 상황

완료된 작업

- 요구사항 정의
- 스키마 설계
- ERD 작성
- 프로젝트 설계
- 멘토님 피드백 반영

진행 중 / 예정

- API 구현
- 인증/인가 구현
- 핵심 기능 개발

▼ 다음 단계 계획

- api 및 인증/인가 구현